

CASO 1: `fetch(..., { mode: "no-cors" })`
¿Funciona? No para leer los datos.

Por qué: `mode: "no-cors"` le dice al navegador que envíe la petición ignorando CORS, pero a cambio privilegia la seguridad opacando la respuesta. El script recibe un objeto `Response` vacío (tipo `opaque`), por lo que `r.json()` va a fallar catastróficamente con un error de parseo.

CASO 2: `Access-Control-Allow-Origin: * + credentials: "include"`
¿Qué sale mal y por qué? Fallo de seguridad por incompatibilidad.

Por qué: Las especificaciones de CORS prohíben estrictamente usar el comodín `*` si la petición incluye credenciales (cookies, tokens de autenticación). El navegador bloqueará la respuesta inmediatamente. Para que funcione con credenciales, el servidor debe responder con el origen explícito: `Access-Control-Allow-Origin: http://localhost:3000`.

CASO 3: `DELETE + Header Authorization` sin manejar `OPTIONS`
¿Qué error aparece en la consola? `Access to fetch at '...' from origin '...' has been blocked by CORS policy: Response to preflight request doesn't pass access control check: It does not have HTTP ok status. (o Method Not Allowed)`.

Por qué: Al usar un método no simple (`DELETE`) y un header personalizado (`Authorization`), el navegador envía automáticamente una petición de verificación previa llamada `Preflight (HTTP OPTIONS)`. Si el servidor no sabe procesar ni responder con un estado exitoso (`200 OK` o `204 No Content`) a ese método `OPTIONS`, el navegador asume que no tiene permisos y aborta la petición real.